

Análise Completa do Processo: “Grátis: Substitua Ferramentas Pagas por Open Source”

Data: 2026-08-20 **Obra:** gratis-open-source (livro + playbook) **Modelo:** free-program (orquestração), subagentes leaf **Fluxo:** /produzir-obra-completa sem campanha e sem máquina (V5)

1. Resumo da Execução

Métrica	Valor
Capítulos	8 (EITA-V2, 7 seções numeradas)
Caracteres totais	~200.000
Referências por capítulo	16–37 (mínimo 16)
Diagramas Mermaid	8 (1 por capítulo)
Blocos de código	17 verificáveis (100%)
Páginas (livro)	98
PDF livro	1.537 KB
PDF playbook	364 KB (28 pág)
Auditoria	CONFORME (R1–R15 + 5 gates F1/F2)
Validar-código	100% (17/17 blocos)
Validar-artefatos	2/2 PDFs abrem
Coleção	2 membros sincronizados
Pacote distribuição	1.901 KB
Commit	a48dbc8 (pushed to origin/main)

2. Gaps Identificados

Gap 1: Subagentes com modelo free-program travam em geração longa

- **Sintoma:** 4 subagentes despachados para caps 5–8; após ~35 min nenhum .md final escrito. Drafts criados (4–12 KB), mas escrita do capítulo completo (~22k chars) não progredia.
- **Causa raiz:** Modelo free instável/lento para geração sustentada de texto longo. Subagente cap 7 entrou em rabbit hole debugando mermaid-cli (problema de ambiente, não do capítulo).
- **Impacto:** Bloqueio total do lote 2. Contorno: escrita manual dos 3 capítulos pelo orquestrador.
- **Evidência:** Delegação deleg_235c0ce5 — task 1 (cap 5) completou em 2202s; tasks 2/3/4 (caps 6/7/8) interrompidas após 2438s sem resposta do modelo.

Gap 2: Mermaid-cli falha localmente por BOM no config do Puppeteer

- **Sintoma:** SyntaxError: Unexpected token ' ' ao tentar renderizar diagramas. mmdc não inicia browser.
- **Causa raiz:** Arquivo de config do Puppeteer (em node_modules) salvo com BOM UTF-8. Lilconfig tenta parsear como JSON e falha.
- **Impacto:** Diagramas não renderizam em PNG localmente. Auditor aceita blocos mermaid como código válido (estrutura OK), mas PDF final não tem figuras renderizadas.
- **Workaround:** Manter blocos mermaid no markdown; compilação via Pandoc+Typst processa como código.

Gap 3: Capa gerada com metadados incompletos (fallback errado)

- **Sintoma:** Capa saiu amarela (#f0b429), categoria “A Oficina Digital”, sem `edition_tag`, subtítulo genérico.
- **Causa raiz:** `config_obra.json` não tinha `cor_primaria`, `subtitulo`, `edition_tag`. Script usa fallback do `sumario_macro.motivo_condutor.nome` para categoria e cor padrão da série.
- **Impacto:** Capa visualmente divergente do padrão da fábrica (livro `git-github-submodules` usa roxo/azul #58a6ff, “A Linha de Montagem”, v1.0 · 2026).
- **Fix aplicado:** Adicionados `cor_primaria`, `subtitulo`, `edition_tag`, `estilo_tecnica` no `config_obra.json`; título abreviado no `sumario_macro.json`.

Gap 4: Título longo demais para validador de capa (3 linhas → erro)

- **Sintoma:** [AVISO] `titulo` viola a regra de quebra de linha: 3 linhas (maximo 2)
- **Causa raiz:** Título original “Grátis: Substitua Ferramentas Pagas por Open Source de Verdade” não cabe na caixa de 1440px com Inter Black 72pt.
- **Fix aplicado:** Abreviado para “GRÁTIS: SUBSTITUA FERRAMENTAS PAGAS POR OPEN SOURCE” (2 linhas).

Gap 5: Citações órfãs no cap 5 (referências globais vs. locais)

- **Sintoma:** R14 flagou 6 citações órfãs [112][114][118][119][123][124] no corpo do cap 5 que não existiam na seção 7 local (1–26).
- **Causa raiz:** Cap 5 escrito por subagente usou numeração global do dossiê (102–124) no corpo, mas seção 7 renumerada localmente.
- **Fix:** Reescrita manual mapeou para refs locais existentes.

Gap 6: Blocos bash não executáveis marcados como tal

- **Sintoma:** `validar-codigo --executar` falhou em 2 blocos (`sudo apt install`, `script restic`).
- **Causa raiz:** Validador executa blocos `bash/sh/python/js`. Comandos que requerem `sudo/ambiente` específico falham.
- **Fix:** Troca de linguagem para `console` (não executável) preservando conteúdo.

Gap 7: Pool de capítulos não avisa sobre rascunhos órfãos

- **Sintoma:** `pool-capitulos.py --status` mostrava 4/8 concluídos, 4 pendentes — mas não indicava que drafts 5–8 existiam sem `.md final`.
- **Gap:** Falta visibilidade de “draft criado, md pendente” no status.

Gap 8: `compilar-para-pdf.py` não resolve slug do hub corretamente sem prefixo `livros/`

- **Sintoma:** `python compilar-para-pdf.py gratis-open-source` → [SKIP] nenhum capítulo ou `livro_final.md` encontrado
- **Causa:** Script procura em `DIR_RAIZ/slug` (layout plano) mas estrutura é `hub: output/gratis-open-source/livros/capitulos/`.
- **Fix:** Usar slug com prefixo do tipo: `livros/gratis-open-source`.

3. Sugestões de Melhoria

Curto Prazo (Próxima Sprint)

#	Melhoria	Esforço	Valor
1	Fallback automático de subagente travado: monitor de heartbeat no <code>delegate_task</code> ; se >10 min sem tool call, cancelar e reescrever localmente	Médio	Alto (evita bloqueio de 35+ min)
2	Validação de citações no subagente: subagente-redator-capitulo deve checar <code>len(refs_secao7) >= min_refs</code> antes de registrar sucesso	Baixo	Alto (evita R14 pós-escrita)
3	Mermaid-cli robusto: wrapper que detecta BOM no config e reescreve sem BOM antes de chamar <code>mmdc</code>	Baixo	Médio (habilita diagramas no PDF)
4	Config obrigatório para capa: <code>config_obra.json</code> deve exigir <code>cor_primaria</code> , <code>subtitulo</code> , <code>edition_tag</code> na entrevista / esbocar	Baixo	Alto (capa correta na 1ª vez)
5	Título validado na entrevista: /esbocar roda <code>validar-capa-texto.py</code> no título proposto; rejeita se >2 linhas	Baixo	Alto
6	Pool status enriquecido: mostrar <code>drafts_pendentes</code> , <code>md_faltando</code> , <code>tempo_medio_escrita</code>	Baixo	Médio
7	Slug resolver no compilar: <code>compilar-para-pdf.py</code> tenta <code>tipos_obra.dir_obra(slug)</code> antes de fallback plano	Baixo	Alto

Médio Prazo (V5.5)

#	Melhoria	Esforço	Valor
8	Subagente especializado em escrita longa: subagente-redator-capitulo-longo com prompt otimizado para chunking (escreve seção por seção, valida cada uma)	Alto	Alto (resolve free-program)
9	Modo “orquestrador escreve”: flag --local-write no /criar-livro que pula subagentes e usa template EITA-V2 preenchido pelo arquiteto	Médio	Alto (determinístico)
10	Cache de diagramas mermaid: renderizar-diagramas.py salva PNGs em imagens/mermaid/ keyed by hash do bloco; reusa entre compilações	Médio	Médio
11	Validação cross-capítulos: auditor checa numeração de refs consistente entre caps (global vs local)	Médio	Alto
12	Playbook auto-valida: extrair-passos-praticos roda validar-playbook.py após montar; falha bloqueia sincronização	Baixo	Médio

Longo Prazo (V6 / Arquitetura)

#	Melhoria	Esforço	Valor
13	Modelo híbrido free/paid: roteamento inteligente – free-program para planejamento/ estratégia, modelo pago (Claude/GPT) para geração de capítulos longos	Alto	Crítico
14	Pipeline assíncrono com checkpoints: cada fase (pesquisa, sumário, lote caps, auditoria, compilação) é step com estado persistido; retoma de qualquer ponto	Alto	Alto
15	Observabilidade completa: métricas por capítulo (tempo, tokens, retries, validações) exportadas para relatorios/ <code>metricas_<slug>.json</code>	Médio	Alto

4. Plano de Ação Completo

Fase 1 — Estabilização Imediata (1–2 dias)

1. **Patch 1:** wrapper mermaid-cli que limpa BOM do config puppeteer (`renderizar-diagramas.py`)
2. **Patch 2:** `validar-capa-texto.py` integrado no `/esbocar` (rejeita título >2 linhas)
3. **Patch 3:** `config_obra.json` schema exige `cor_primaria`, `subtitulo`, `edition_tag`
4. **Patch 4:** `subagente-redator-capitulo.md` adiciona auto-validação R4/R14 antes de `--registrar`
5. **Patch 5:** `pool-capitulos.py --status` mostra `drafts_pendentes + md_faltando`
6. **Patch 6:** `compilar-para-pdf.py` tenta `dir_obra(slug)` antes de `DIR_RAIZ/slug`
7. **Teste:** rodar fluxo completo em livro novo (ex.: “Docker para Devs”) e confirmar 0 intervenção manual

Fase 2 — Subagentes Confiáveis (3–5 dias)

1. Criar `subagente-redator-capitulo-longo.md` com estratégia chunked:
 - Input: `draft.json` + dossiê
 - Loop: para cada seção EITA (1–7): escrever → validar-código local → validar refs → registrar progresso
 - Output: `cap_NN.md` completo + registro pool
2. Adicionar heartbeat no `delegate_task` (poll a cada 2 min; se 3 polls sem `tool_call` → steer “escreva próxima seção AGORA”)
3. Fallback automático: se subagente falha 2x, orquestrador assume escrita daquela seção
4. Benchmark: comparar tempo/qualidade free-program vs paid para caps 5–8

Fase 3 — Orquestração Determinística (1 semana)

1. Flag `--modo=local` em `criar-livro.md`: pula subagentes, usa `pool-capitulos --plano --lote 1` com execução sequencial local
2. Script orquestrador único (`produzir-obra-local.py`) que encadeia: `minerar-fontes` → `indexar-dossie` → `arquiteto (sumário)` → `loop caps (redator local)` → `auditar` → `compilar` → `playbook` → `empacotar`

3. Cada step grava checkpoint em estado SQLite (db_state); retoma automática em falha
4. Métricas por step: tokens, tempo, validações, retries → relatorios/metricas_<slug>.json

Fase 4 — Híbrido Free/Paid (2 semanas)

1. Config delegation.models: { "planning": "free-program", "writing": "claude-3.5-sonnet", "audit": "free-program" }
2. Router no delegate_task: lê task.type → seleciona modelo
3. Orçamento por obra: max_tokens_writing = tamanho_obra * 1.5; alerta se >80%
4. Fallback paid→free se quota excedida (degradação graciosa)

Fase 5 — Observabilidade e Qualidade Contínua

1. Dashboard simples (HTML estático em relatorios/) lendo metricas_*.json
2. Alertas: R2 < 90%, R4 < 100%, tempo_caps > 30min, mermaid_fail_rate > 10%
3. Regressão nightly: roda fluxo completo em livro-teste pequeno; falha = issue automática

5. Métricas de Sucesso (KPIs)

Métrica	Atual (free-only)	Alvo (híbrido)
Tempo total livro M (8 caps)	~4h (com bloqueios)	<45 min
Intervenção manual / livro	3–4 (caps, capa, refs, mermaid)	0
Taxa sucesso subagentes caps 5–8	0/4 (free-program)	4/4 (paid writing)
Diagramas no PDF final	0% (mermaid quebrado)	100%
Capa correta na 1ª compilação	Não (fallback errado)	Sim
Auditoria CONFORME 1ª passada	Sim (após fixes manuais)	Sim
Commits por livro	2–3 (fixes incrementais)	1 (atômico)

6. Decisões Arquiteturais

1. **Não corrigir mermaid-cli no sistema** — wrapper local no script renderizar-diagramas.py é suficiente e isolado.
2. **Manter subagentes como padrão** — mas com fallback determinístico automático. O modelo free serve para planejamento/estratégia; escrita longa vai para modelo pago.
3. **Validação “shift-left”**: o máximo possível de gates (R4, R14, capa, título) roda *durante* a escrita, não depois.
4. **Estado persistido em SQLite (db_state)** — cada fase checkpointada; retoma é feature, não exceção.
5. **Métricas são cidadãs de primeira classe** — todo fluxo emite metricas_<slug>.json; dashboard gera relatório de saúde da fábrica.

7. Próximos Passos Imediatos

1. **Aplicar Patches 1–6** (lista Fase 1) — ~2h de trabalho

2. **Teste de regressão:** criar livro pequeno (“Guia Rápido Markdown”) com fluxo completo automatizado
3. **Documentar** em docs/operacao/fluxo-v5.5.md os novos comandos e flags
4. **Agendar** Fase 2 (subagente chunked) para próxima semana

8. Conclusão

O fluxo V5 funciona e entrega material de qualidade (CONFORME em todas as validações), mas depende de intervenção manual pesada quando modelo free trava. A transição para **orquestração híbrida free/paid + checkpoints + validação shift-left** elimina os gargalos e torna a fábrica previsível e escalável.